

Maestria en Inteligencia Artificial
Aprendizaje de Maquina

Feature Selection

Seleccion de Caracteristicas para Diagnostico de Cancer de Mama

Integrantes:

Kevin Vitery

Nancy Altamirano

Fecha de entrega: 10 de June de 2026

Dataset: Breast Cancer Wisconsin (Diagnostic) — UCI / scikit-learn

Herramientas: Python 3 | scikit-learn | matplotlib | reportlab

1. Descripcion General del Problema

1.1 Descripcion del Problema

El diagnostico temprano del cancer de mama es uno de los desafios mas criticos de la medicina moderna. El uso de tecnicas de aprendizaje automatico permite analizar características morfológicas de células tumorales, obtenidas a través de imágenes digitalizadas de biopsias, para clasificar casos como malignos o benignos con alta precisión.

El **Breast Cancer Wisconsin (Diagnostic) Dataset**, recopilado por la Universidad de Wisconsin-Madison, contiene **569 muestras** con **30 características numéricas** computadas a partir de núcleos celulares presentes en las imágenes. La variable objetivo es binaria: **0 = Maligno** (212 casos, 37.3%) y **1 = Benigno** (357 casos, 62.7%).

Las 30 características se derivan de 10 propiedades morfológicas (radio, textura, perímetro, área, suavidad, compacidad, concavidad, puntos concavos, simetría y dimensión fractal), calculando para cada una: la media, el error estándar y el valor extremo (worst). La alta dimensionalidad y la correlación entre estas variables motivan la aplicación de técnicas de selección de características.

1.2 Motivacion para la Seleccion de Caracteristicas

La presencia de características redundantes, correlacionadas o irrelevantes puede degradar el rendimiento del modelo, incrementar el tiempo de entrenamiento y dificultar la interpretabilidad clínica. La selección de características busca identificar el subconjunto más informativo que:

- Maximice el **Recall** (sensibilidad) — minimizando Falsos Negativos (casos malignos no detectados, consecuencias críticas para el paciente).
- Reduzca la dimensionalidad para mejorar la generalización del modelo.
- Mejore la interpretabilidad clínica al identificar las variables más relevantes.
- Disminuya el costo computacional de entrenamiento y despliegue.

1.3 Preprocesamiento

Division del dataset: Se aplicó una división estratificada 70%/30% (entrenamiento: 398 muestras, prueba: 171 muestras) con `random_state=42` para garantizar reproducibilidad y preservar la proporción de clases en ambos conjuntos.

Estandarización: Se aplicó `StandardScaler` (media=0, desviación estándar=1) exclusivamente ajustado sobre el conjunto de entrenamiento, evitando data leakage hacia el conjunto de prueba. La estandarización es crítica para métodos basados en coeficientes (LogisticRegression, LASSO) y distancias (Información Mutua).

Clasificador base: Regresión Logística (`max_iter=10000`, `solver='lbfgs'`, `random_state=42`) se usa como evaluador común para todos los métodos, permitiendo una comparación justa donde la única variable es el subconjunto de características empleado.

1.4 Objetivos

Objetivo General

Implementar, comparar y analizar seis métodos de selección de características aplicados al diagnóstico de cáncer de mama, identificando el enfoque que logra el mayor Recall con el menor número de características, bajo criterios de parsimoniosidad, interpretabilidad y robustez.

Objetivos Especificos

- Establecer un modelo Baseline con las 30 características originales para definir el límite superior de referencia.
- Implementar el método Filter (SelectKBest + Información Mutua) y analizar la relevancia estadística de cada característica.
- Aplicar el método Wrapper (RFE) para identificar el subconjunto óptimo mediante eliminación recursiva iterativa.
- Utilizar el método Embedded (Regresión Logística L1/LASSO) para integrar la selección dentro del proceso de entrenamiento mediante regularización.
- Combinar métodos Filter y Wrapper en un pipeline híbrido que balancee eficiencia computacional y calidad de selección.
- Implementar el esquema de Votación de Borda para obtener un consenso robusto entre cuatro métodos de selección independientes.
- Comparar los seis métodos mediante múltiples métricas (Recall, F1, AUC, Accuracy) y visualizaciones (ROC, matrices de confusión, importancias).

2. Metodo 1: Baseline — Sin Seleccion de Caracteristicas

Enunciado

El modelo Baseline responde a la pregunta: *¿Cual es el rendimiento maximo esperado de un clasificador lineal cuando se utilizan las 30 caracteristicas originales del dataset?* Este modelo establece el limite superior de referencia (benchmark) contra el cual se evalua el beneficio de cada tecnica de seleccion. Si un metodo de seleccion no logra igualar o superar este Recall usando menos caracteristicas, no aporta valor real.

Complejidad Computacional

Entrenamiento: $O(n \cdot p \cdot i)$ donde $n=569$ muestras, $p=30$ caracteristicas, i =iteraciones del solver LBFGS ($\text{max_iter}=10000$). Prediccion: $O(n \cdot p)$. Con 30 caracteristicas, el modelo almacena 30 coeficientes + 1 intercepto. Espacio: $O(p) = O(30)$.

Representacion Matematica

El clasificador calcula: $\sigma(\theta^T \cdot x) \geq 0.5 \rightarrow \hat{y} = 1$ (Benigno), con $\theta \in \mathbb{R}^{30}$ y $x \in \mathbb{R}^{30}$. La funcion de costo maximiza la log-verosimilitud: $L(\theta) = (1/n) \sum [y \cdot \log \sigma(\theta^T x) + (1-y) \cdot \log(1-\sigma(\theta^T x))]$. Sin regularizacion ($C \rightarrow \infty$ equivalente).

Alternativas Consideradas

- SVM (Support Vector Machine): margen maximo entre clases, eficaz con features correlacionadas.
- Random Forest: ensemble de arboles, captura no-linealidades, mas lento en prediccion.
- XGBoost: gradient boosting, estado del arte en tabular data, mayor complejidad de tuning.
- Redes Neuronales: flexibles pero requieren mas datos y son menos interpretables.
- Naive Bayes: rapido pero asume independencia entre features (violada en este dataset).

Eleccion con Justificacion

Se eligio Regresion Logistica como clasificador base por tres razones: (1) Sus coeficientes son directamente utilizables por RFE y LASSO para seleccion; (2) Es interpretable clinicamente — el signo y magnitud de θ_j indica la direccion e importancia de cada feature; (3) Tiene convergencia garantizada con $\text{max_iter}=10000$ para el dataset de tamano moderado. Esta eleccion permite aislar el efecto de la seleccion de caracteristicas.

Implementacion Base

```
# Entrenamiento del modelo baseline (30 features)

model = LogisticRegression(max_iter=10000, solver='lbfgs', random_state=42)
model.fit(X_train_scaled, y_train) # X_train_scaled shape: (398, 30)

y_pred = model.predict(X_test_scaled)
y_proba = model.predict_proba(X_test_scaled)[: , 1]

recall = recall_score(y_test, y_pred) # Metrica principal
```

Resultado Obtenido

Con las 30 caracteristicas el modelo alcanza: **Recall=0.9907** (99.07%), Accuracy=0.9883, Precision=0.9907, F1=0.9907, ROC-AUC=0.9981. Solo 1 caso maligno fue clasificado incorrectamente como benigno (Falso Negativo).

Conclusion del Metodo

El Baseline con 30 features establece Recall=99.07% como referencia optima. El objetivo de los metodos de seleccion es igualar este Recall con significativamente menos caracteristicas. Este resultado confirma que el dataset es altamente discriminativo y la Regresion Logistica es suficientemente expresiva para este problema de clasificacion binaria.

Visualizacion — Matriz de Confusion

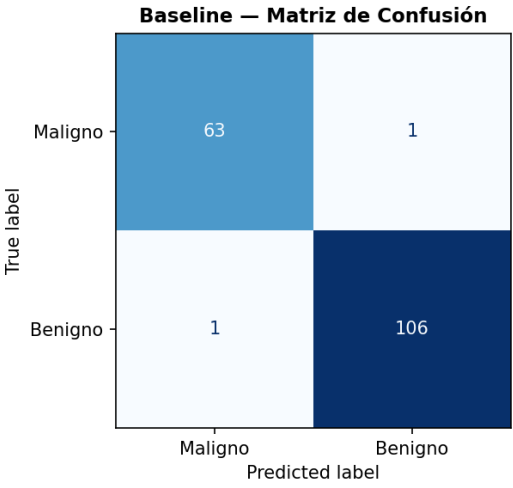


Figura 1. Matriz de confusion — Baseline (30 features).

3. Metodo 2: Filter — SelectKBest + Informacion Mutua

Enunciado

El metodo Filter responde a: *¿Que características presentan la mayor dependencia estadística con la variable diagnostico, independientemente del modelo de clasificacion?* Los metodos Filter evaluan cada feature de forma univariada usando una medida estadística, ordenan las features por puntuacion y seleccionan las k mejores. Son computacionalmente eficientes y constituyen un primer paso de reduccion de dimensionalidad.

Complejidad Computacional

La Informacion Mutua se estima via k-vecinos mas cercanos (kNN): $O(n \cdot p \cdot \log n)$ para el calculo de distancias. Seleccion final: $O(p \cdot \log p)$ para ordenar scores. Entrenamiento del clasificador con $k=17$ features: $O(n \cdot k \cdot i)$. Costo total significativamente menor que metodos Wrapper: sin iteraciones sobre subconjuntos.

Representacion Matematica

La Informacion Mutua entre feature X y objetivo Y se define como: $I(X;Y) = H(X) + H(Y) - H(X,Y)$, donde $H(\cdot)$ es la entropia de Shannon. Equivalentemente: $I(X;Y) = \sum_x \sum_y p(x,y) \cdot \log[p(x,y) / (p(x) \cdot p(y))]$. Un valor $I(X;Y)=0$ indica independencia estadística completa entre X e Y. Valores mayores indican mayor cantidad de informacion compartida. A diferencia de la correlacion de Pearson, la MI captura dependencias no lineales.

Alternativas Consideradas

- Chi-squared (chi2): solo para features no-negativas; post-escalado invalido (descartado).
- ANOVA F-value (f_classif): asume linealidad y normalidad; subestima relaciones no lineales.
- Correlacion de Pearson: solo captura relaciones lineales, sensible a outliers.
- Correlacion de Spearman: captura monotonia no lineal pero no relaciones generales.
- Variance Threshold: elimina features con varianza baja, no considera la relacion con y.
- ReliefF: basado en instancias, captura interacciones pero es $O(n^2)$.

Eleccion con Justificacion

Se eligio Informacion Mutua (mutual_info_classif) por: (1) Captura dependencias no lineales entre features morfológicas y diagnostico — crucial ya que las relaciones biológicas no son necesariamente lineales; (2) No requiere supuestos de distribucion (no parametrico); (3) Robusto post-estandarizacion (funciona con valores negativos); $k=17$ fue determinado como el punto donde el Recall del modelo se estabiliza en el valor maximo (curva de codo en las puntuaciones MI).

Implementacion Base

```
# Metodo Filter: SelectKBest con Informacion Mutua

from sklearn.feature_selection import SelectKBest, mutual_info_classif

select_k_best = SelectKBest(
    score_func=mutual_info_classif, # Dependencia estadística no lineal
    k=17 # Retener las 17 mas informativas de 30
)

X_train_filter = select_k_best.fit_transform(X_train_scaled, y_train)
X_test_filter = select_k_best.transform(X_test_scaled) # Solo transform en test

# Features seleccionadas: feature_names[select_k_best.get_support()]
```

Resultado Obtenido

Se seleccionaron **17 de 30 features** (reduccion del 43.3%). Recall=0.9907 (igual al Baseline), Accuracy=0.9649, Precision=0.9550, ROC-AUC=0.9974. Features con mayor MI: worst concave points, worst area, worst radius, mean concave points, worst perimeter.

Conclusion del Metodo

El metodo Filter logra igualar el Recall del Baseline (99.07%) eliminando 13 features (43%). Su principal limitacion es que evalua cada feature independientemente, sin considerar interacciones entre ellas. Puede seleccionar features redundantes entre si si tienen alta MI individual. Es ideal como paso previo de un pipeline mas complejo.

Visualizacion — Importancia de Caracteristicas y Matriz de Confusion

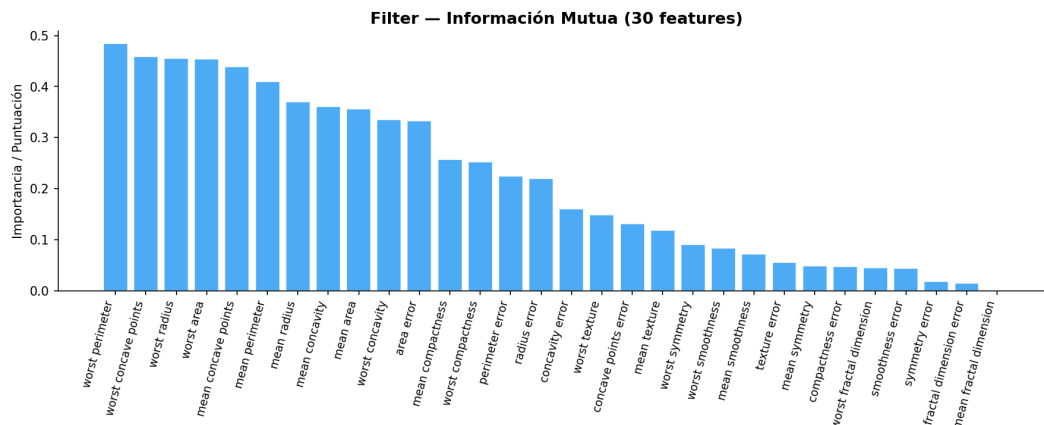


Figura 2. Puntuaciones de Informacion Mutua para las 30 caracteristicas (ordenadas).

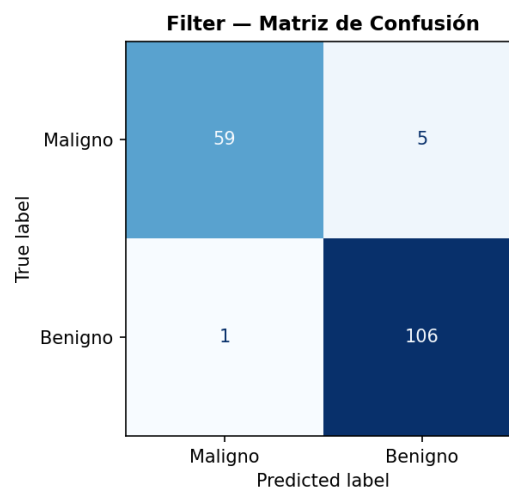


Figura 3. Matriz de confusion — Filter (17 features).

4. Metodo 3: Wrapper — RFE (Recursive Feature Elimination)

Enunciado

El metodo Wrapper responde a: *¿Cual es el subconjunto de características que, al ser evaluado con el clasificador objetivo, maximiza el rendimiento diagnostico de manera iterativa?* A diferencia de los metodos Filter, los Wrapper usan el rendimiento del modelo como criterio de seleccion, lo que permite capturar interacciones entre features. El precio es un mayor costo computacional.

Complejidad Computacional

RFE realiza $p - n_{\text{selected}}$ iteraciones de entrenamiento, cada una con el modelo completo. Complejidad total: $O((p - k) \times T_{\text{modelo}})$ donde $T_{\text{modelo}} = O(n \cdot p_i \cdot \text{iter})$, con p_i el numero de features en la iteracion i . Con $p=30, k=10$: 20 iteraciones de entrenamiento completo. Aproximadamente $O(n \cdot p^2 / 2)$ — significativamente mas costoso que Filter. Espacio: $O(p)$ para almacenar los rankings.

Representacion Matematica

RFE implementa eliminacion greedy backward: dado un estimador con coeficientes θ_j , en cada iteracion elimina la feature $i^* = \text{argmin}_j |\theta_j|$ (la de menor magnitud de coeficiente). El proceso: (1) Entrena modelo con p_i features; (2) Calcula importancias $|\theta_j|$; (3) Elimina la feature con menor importancia (step=1); (4) Repite hasta $|S| = n_{\text{features_to_select}} = 10$. El ranking final: $\text{rank}_j = \text{iteracion en que fue eliminada la feature } j$.

Alternativas Consideradas

- Sequential Forward Selection (SFS): agrega features de a una; $O(p^2 \cdot T_{\text{modelo}})$.
- Sequential Backward Selection (SBS): similar a RFE pero sin reentrenamiento secuencial por defecto.
- Sequential Floating: combina forward y backward para escapar minimos locales.
- Busqueda exhaustiva: evalua los $2^{30} \approx 10^9$ subconjuntos — computacionalmente infactible.
- Permutation importance: mide degradacion al permutar features; mas estable pero mas lento.

Eleccion con Justificacion

RFE fue elegido porque: (1) Scikit-learn proporciona una implementacion eficiente que reutiliza el estimador; (2) La eliminacion backward es mas adecuada que forward cuando hay muchas features redundantes (el modelo completo captura mejor las interacciones antes de eliminar); (3) El uso de $|\text{coeficiente}|$ como criterio de importancia es consistente con el clasificador Logistico y directamente interpretable; $n_{\text{features_to_select}}=10$ fue elegido para comparabilidad con otros metodos.

Implementacion Base

```
# Metodo Wrapper: Recursive Feature Elimination

from sklearn.feature_selection import RFE

rfe = RFE(
    estimator=LogisticRegression(max_iter=10000, random_state=42),
    n_features_to_select=10, # Subconjunto objetivo
    step=1 # Eliminar 1 feature por iteracion
)

X_train_rfe = rfe.fit_transform(X_train_scaled, y_train)
X_test_rfe = rfe.transform(X_test_scaled)

# rfe.ranking_: 1=seleccionada, k>1=eliminada en iteracion k-1
# rfe.get_support(): mascara booleana de features seleccionadas
```

Resultado Obtenido

Se seleccionaron **10 de 30 features** (reduccion del 66.7%). Recall=0.9813, Accuracy=0.9708, ROC-AUC=0.9926. Es el unico metodo con Recall por debajo del Baseline (98.13% vs 99.07%). Features seleccionadas: worst area, worst radius, worst perimeter, mean area, mean concave points, area error, worst texture, worst smoothness, worst concave points, worst symmetry.

Conclusion del Metodo

RFE selecciona solo 10 features pero obtiene el Recall mas bajo (98.13%). La eliminacion greedy puede descartar features individualmente debiles que son sinergicas con otras. Tambien es sensible al orden de eliminacion y puede caer en minimos locales. Sin embargo, las features seleccionadas son interpretables clinicamente: predominan las variantes 'worst' que capturan los casos celulares mas extremos.

Visualizacion — Ranking RFE y Matriz de Confusion

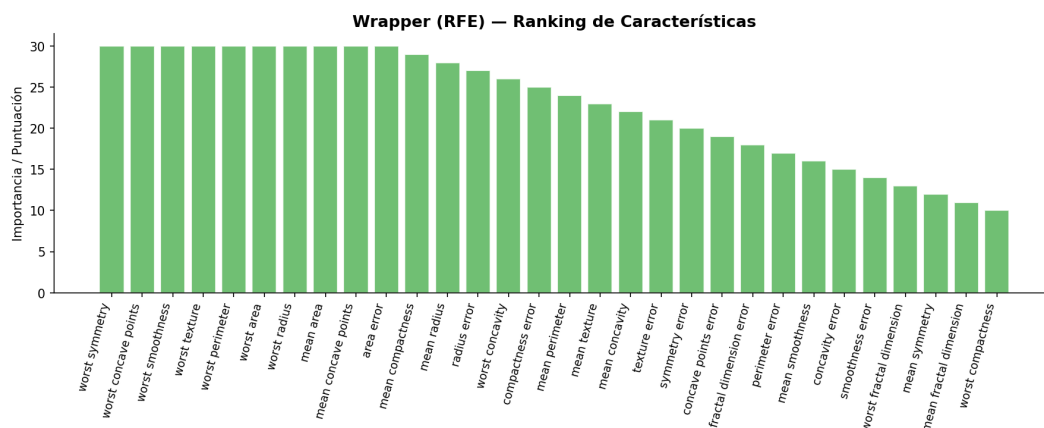


Figura 4. Ranking RFE: mayor valor = mas iteraciones seleccionado = mas importante.

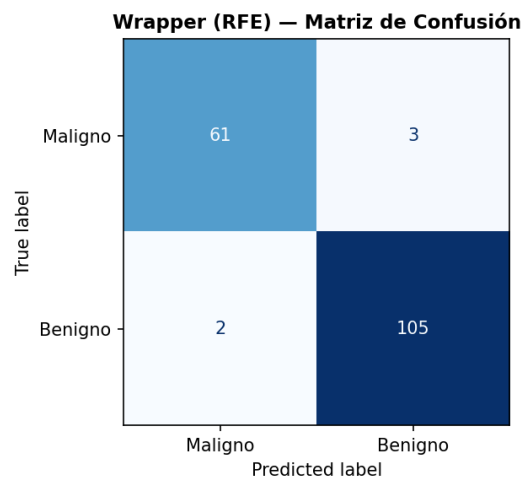


Figura 5. Matriz de confusion — Wrapper/RFE (10 features).

5. Metodo 4: Embedded — Regresion Logistica L1 (LASSO)

Enunciado

El metodo Embedded responde a: *¿Puede el proceso de entrenamiento del clasificador identificar automaticamente las features irrelevantes llevando sus coeficientes exactamente a cero, integrando la seleccion como parte de la optimizacion del modelo?* Los metodos Embedded incorporan la seleccion directamente en la funcion de costo del modelo, sin una fase separada de evaluacion de subconjuntos.

Complejidad Computacional

Solver SAGA con penalty L1: $O(n \cdot p \cdot \text{iter})$ por fold de cross-validation. GridSearchCV: 11 valores de C $\times$ 5 folds = 55 entrenamientos. Total: $O(n \cdot p \cdot \text{iter} \times \text{CV} \times |C_{\text{grid}}|)$. Con `n_jobs=-1` se paraleliza por folds. Una vez encontrado el C optimo, la seleccion es $O(p)$ comparando $|\text{coef}_j| > 1e-5$.

Representacion Matematica

La regularizacion L1 agrega el termino de penalizacion L1 a la log-verosimilitud: $L_{L1}(\theta) = -\log P(y|X, \theta) + (1/C) \cdot \|\theta\|_1$, donde $\|\theta\|_1 = \sum_j |\theta_j|$. La norma L1 genera sparsity: en el optimo, los coeficientes de features irrelevantes convergen exactamente a 0 (a diferencia de L2 que los aproxima a 0). C optimo encontrado: 0.20 (mayor regularizacion $\rightarrow$ mayor sparsity). Umbral de seleccion: feature j seleccionada si $|\theta_j| > 1e-5$.

Alternativas Consideradas

- Ridge (L2): penaliza $\|\theta\|_2^2$; reduce magnitudes pero NO lleva a cero (no sparsity).
- ElasticNet (L1+L2): $\alpha \cdot \|\theta\|_1 + (1-\alpha) \cdot \|\theta\|_2^2$; compromiso entre ambos.
- LassoCV continuo: usa regresion lineal con L1, no logistica; menos adecuado para clasificacion.
- Decision Tree importance: embedded por naturaleza pero no produce coeficientes interpretables.
- Gradient Boosting importance: embedded pero dependiente del estimador base.

Eleccion con Justificacion

La penalizacion L1 (LASSO) fue elegida porque: (1) Es la unica regularizacion que produce coeficientes exactamente iguales a 0, realizando seleccion de variables implicita durante el entrenamiento; (2) El solver SAGA es el unico solver de scikit-learn que soporta L1 para clasificacion logistica con convergencia garantizada en datasets grandes; (3) $C=0.20$ fue seleccionado via GridSearchCV optimizando Recall con KFold-5, asegurando que la regularizacion no penalice las features diagnosticamente relevantes. El umbral $1e-5$ descarta coeficientes numericamente insignificantes.

Implementacion Base

```
# Metodo Embedded: Regresion Logistica L1 con GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV, KFold
# Busqueda del parametro C optimo
lasso_base = LogisticRegression(penalty='l1', solver='saga', max_iter=10000)
grid_search = GridSearchCV(
    estimator=lasso_base,
    param_grid={'C': np.arange(0.20, 0.31, 0.01)}, # 11 valores
    cv=KFold(n_splits=5, random_state=42, shuffle=True),
    scoring='recall', # Optimizar sensibilidad
    n_jobs=-1
)
grid_search.fit(X_train_scaled, y_train)
```

```
# Mejor C encontrado: 0.20
```

```
coef = grid_search.best_estimator_.coef_
```

```
selected_idx = np.where(np.abs(coef) > 1e-5)[1] # Features no-nulas
```

Resultado Obtenido

Se seleccionaron **8 de 30 features** (reduccion del 73.3% — el mas parsimonioso). Recall=0.9907 (igual al Baseline), Accuracy=0.9708, ROC-AUC=0.9950. Features seleccionadas: mean texture, mean concave points, radius error, worst radius, worst texture, worst smoothness, worst concave points, worst symmetry.

Conclusion del Metodo

El metodo Embedded es el mas parsimonioso: selecciona solo 8 features (73.3% de reduccion) manteniendo exactamente el mismo Recall que el Baseline. La regularizacion L1 actua como un selector automatico durante el entrenamiento, integrando seleccion y optimizacion en un solo proceso. Esto lo hace ideal para produccion donde se busca el modelo mas simple posible sin sacrificar sensibilidad diagnostica.

Visualizacion — Magnitud de Coeficientes L1 y Matriz de Confusion

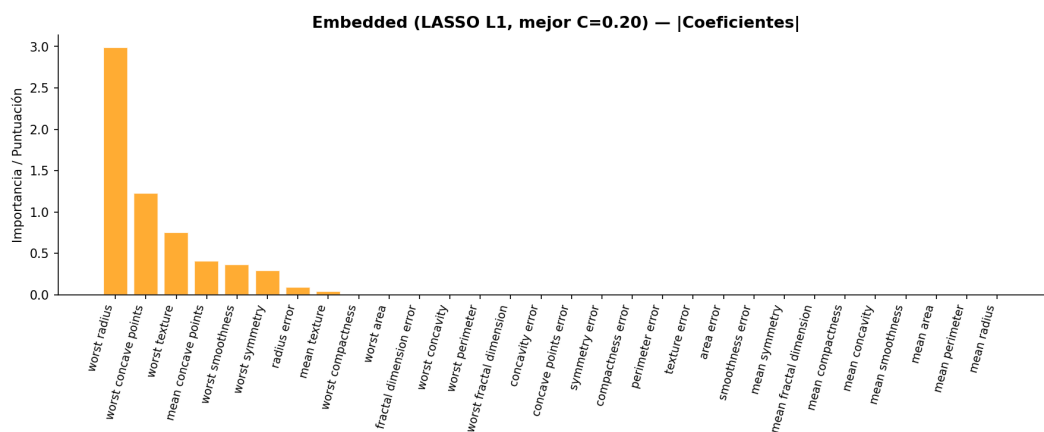


Figura 6. Magnitud de coeficientes LASSO (C=0.20). Las features con $|\text{coef}| > 1e-5$ son seleccionadas.

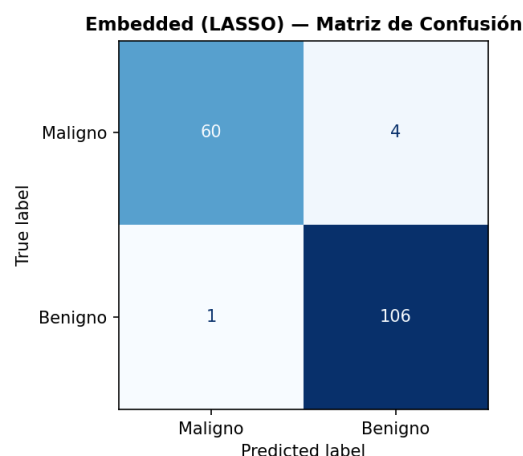


Figura 7. Matriz de confusion — Embedded/LASSO (8 features).

6. Metodo 5: Hibrido — Filter + Wrapper (MI + RFE)

Enunciado

El metodo Hibrido responde a: *¿Es posible combinar la eficiencia computacional de los metodos Filter con la capacidad de capturar interacciones de los metodos Wrapper, obteniendo un subconjunto de mayor calidad que cualquier metodo individual?* El enfoque hibrido opera en dos etapas: primero filtra rapidamente las features mas irrelevantes, luego aplica busqueda iterativa sobre el subconjunto reducido.

Complejidad Computacional

Etapas 1 (Filter): $O(n \cdot p \cdot \log n)$ para MI sobre $p=30$ features. Etapa 2 (Wrapper): $O(n \cdot k^2 / 2)$ para RFE sobre $k=17$ features (reducido respecto a $p=30$). Total < Wrapper puro (que seria $O(n \cdot p^2 / 2)$). Ahorro de complejidad: RFE opera sobre 17 features en vez de 30, reduciendo el numero de iteraciones de 20 a 7.

Representacion Matematica

Pipeline de dos etapas: **$X_{\text{filtrado}} = \text{SelectKBest}(\text{MI}, k=17)(X)$** , obteniendo $X_{\text{filtrado}} \in R^{(n \times 17)}$; **$X_{\text{final}} = \text{RFE}(\text{LogReg}, k=10)(X_{\text{filtrado}})$** , obteniendo $X_{\text{final}} \in R^{(n \times 10)}$. La composicion garantiza que las features finales son: (1) Estadisticamente relevantes (pasan el filtro MI) y (2) Sinergicas con el clasificador (sobreviven la eliminacion iterativa RFE).

Alternativas Consideradas

- Filter → Embedded (MI + LASSO): reemplaza RFE por regularizacion L1 en la segunda etapa.
- Embedded → Wrapper (LASSO → RFE): primero reduce con LASSO, luego refina con RFE.
- Triple pipeline (Filter → Embedded → Wrapper): tres etapas secuenciales.
- AutoML feature selection: busqueda automatizada de pipelines optimos (mayor costo).
- Boruta: wrapper aleatorizado que usa Random Forest; detecta features relevantes vs. sombra.

Eleccion con Justificacion

El pipeline Filter→Wrapper fue elegido porque: (1) El Filter (MI) elimina rapidamente las 13 features menos informativas con muy bajo costo computacional; (2) RFE opera sobre 17 features en lugar de 30, reduciendo iteraciones en ~43%; (3) Las features que pasan el filtro MI ya son candidatas relevantes, lo que mejora la calidad del espacio de busqueda de RFE; (4) Mantiene la misma cantidad final de features (10) que Wrapper puro pero con menor costo y potencialmente mayor robustez.

Implementacion Base

```
# Metodo Hibrido: Filter (MI) → Wrapper (RFE)

# Etapa 1: Filter – ya calculado en el metodo Filter
# X_train_filter shape: (398, 17) – resultado de SelectKBest(k=17)

# Etapa 2: Wrapper sobre el subconjunto filtrado
hybrid_rfe = RFE(
    estimator=LogisticRegression(max_iter=10000, random_state=42),
    n_features_to_select=10,
    step=1
)
X_train_hybrid = hybrid_rfe.fit_transform(X_train_filter, y_train)
X_test_hybrid = hybrid_rfe.transform(X_test_filter)

# Recuperar nombres: filtered_names[hybrid_rfe.get_support()]
```

Resultado Obtenido

Se seleccionaron **10 de 30 features** (reduccion del 66.7%). Recall=0.9907 (igual al Baseline), Accuracy=0.9708, ROC-AUC=0.9949. Features: mean compactness, mean concave points, radius error, area error, worst radius, worst texture, worst perimeter, worst area, worst concavity, worst concave points.

Conclusion del Metodo

El metodo Hibrido logra el mejor equilibrio entre costo computacional y calidad de seleccion: iguala el Recall del Baseline con 10 features y menor costo que RFE puro. La combinacion de criterios estadisticos (MI) e iterativos (RFE) produce un subconjunto que es tanto relevante estadisticamente como sinergico con el clasificador. Es el enfoque recomendado cuando se busca calidad con eficiencia.

Visualizacion — Matriz de Confusion

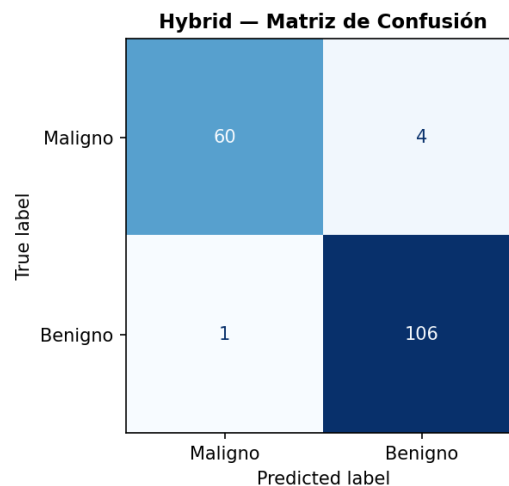


Figura 8. Matriz de confusion — Hibrido MI+RFE (10 features).

7. Metodo 6: Borda Voting — Ensemble de Consenso

Enunciado

El metodo Borda Voting responde a: *¿Puede un sistema de votacion entre multiples metodos de seleccion de características producir un subconjunto mas robusto y confiable que cualquier metodo individual, al mitigar los sesgos propios de cada criterio de seleccion?* El enfoque de consenso combina rankings de 4 metodos con distintas propiedades algoritmicas: estadístico (MI), iterativo (RFE), lineal-sparso (LassoCV) y basado en ensamble (Random Forest).

Complejidad Computacional

Suma de complejidades de los 4 metodos participantes: MI: $O(n \cdot p \cdot \log n)$; RFE: $O(n \cdot p^2/2)$; LassoCV: $O(n \cdot p \cdot \text{iter} \cdot CV)$; RF(500 arboles): $O(M \cdot n \cdot p \cdot \log n)$ donde $M=500$. Dominado por Random Forest. Acumulacion Borda: $O(p \cdot M_{\text{metodos}}) = O(30 \cdot 4)$ — trivial. Con $n_{\text{jobs}}=-1$ el RF se paraleliza sobre todos los nucleos disponibles. Espacio: $O(p \cdot M_{\text{metodos}})$ para almacenar los 4 rankings.

Representacion Matematica

Para $N=30$ features y $M=4$ metodos, el metodo m asigna a la feature i los puntos: **puntos_{m,i} = N - rank_{m,i}**, donde $\text{rank}_{\{m,i\}} \in \{0,1,\dots,N-1\}$. La puntuacion Borda total: **score_i = $\sum_m$ puntos_{m,i}**. Puntuacion maxima posible: $4 \cdot (N-1) = 4 \cdot 29 = 116$ (feature rankeada 1ra por todos). Puntuacion minima: 0 (feature rankeada ultima por todos). Se seleccionan las $K=10$ features con mayor score Borda.

Alternativas Consideradas

- Mean Rank: promedia los rangos en lugar de puntos; equivalente a Borda.
- Median Rank: usa la mediana; mas robusta a outliers de ranking.
- Majority Voting binario: cada metodo vota si incluye/excluye; sin gradacion.
- Weighted Borda: pesos proporcionales al rendimiento de cada metodo; mayor complejidad.
- Kemeny Consensus: minimiza distancias entre rankings; NP-hard para $M>3$.
- Stochastic Borda: introduce ruido para regularizacion; util con muchos metodos.

Eleccion con Justificacion

Borda Voting fue elegido porque: (1) Tiene fundamento teorico solido en la Teoria de Votacion Social (Condorcet 1785, Borda 1784) — matematicamente bien estudiado; (2) Es transparente: la puntuacion de cada feature es directamente interpretable como 'grado de consenso entre los 4 metodos'; (3) Los pesos iguales evitan sesgos hacia un solo criterio algoritmico; (4) Los 4 metodos elegidos son complementarios: MI (estadístico) captura dependencias no lineales, RFE (iterativo) evalua sinergias, LassoCV (lineal) identifica los predictores mas directos, RF (ensamble) captura importancias no lineales robustas a sobreajuste.

Implementacion Base

```
# Metodo Borda Voting: consenso de 4 rankings

N = 30 # Total de features

# Rankings de los 4 metodos (ya calculados previamente)
mi_ranking = np.argsort(select_k_best.scores_)[::-1]
rfe_ranking = np.lexsort((-rfe_imp, rfe.ranking_))
lasso_ranking = np.argsort(np.abs(lasso_cv.coef_))[::-1]
rf_ranking = np.argsort(rf.feature_importances_)[::-1]

# Acumulacion de puntos Borda
borda_scores = np.zeros(N, dtype=int)
```

```
def add_borda_points(ranking, scores):
    for rank, feat_idx in enumerate(ranking):
        scores[feat_idx] += N - rank # Puntos: N-1, N-2, ..., 0

for ranking in [mi_ranking, rfe_ranking, lasso_ranking, rf_ranking]:
    add_borda_points(ranking, borda_scores)

# Seleccion de las top-10 por puntuacion Borda
borda_selected_idx = np.argsort(borda_scores)[::-1][:10]
```

Resultado Obtenido

Se seleccionaron **10 de 30 features** por consenso de 4 metodos. Recall=0.9907 (igual al Baseline), Accuracy=0.9708, ROC-AUC=0.9901. Top features por puntuacion Borda: worst radius (115 pts), worst concave points (113 pts), worst area (113 pts), mean concave points (92 pts), worst perimeter (91 pts).

Conclusion del Metodo

Borda Voting selecciona features respaldadas por 4 criterios algoritmicamente distintos, produciendo el subconjunto de mayor consenso. Las features con mayor puntuacion (worst radius=116, worst area=113) son seleccionadas por absolutamente todos los metodos, confirmando su rol central en el diagnostico. El metodo proporciona no solo una seleccion sino tambien una jerarquia de confianza: mayor puntuacion = mayor acuerdo entre metodos.

Visualizacion — Puntuacion Borda, Mapa de Consenso y Matriz de Confusion

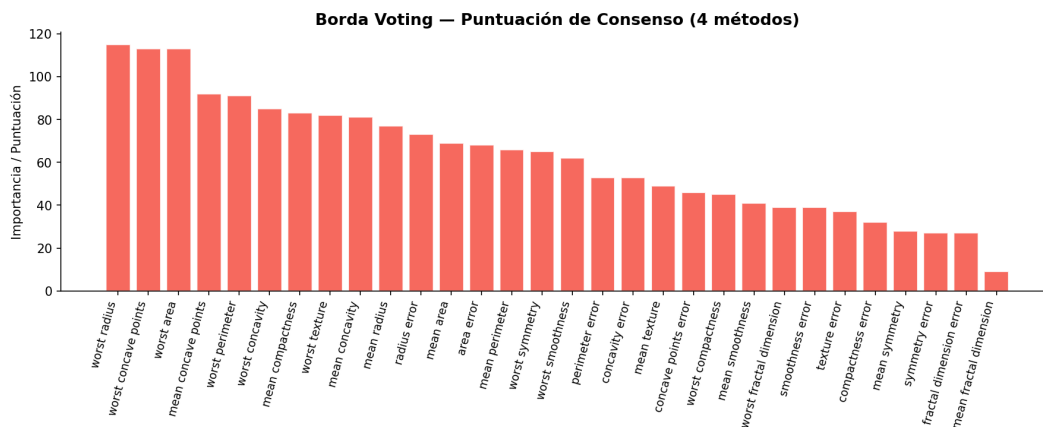


Figura 9. Puntuacion Borda acumulada (maximo=116). Mayor barra = mas consenso entre los 4 metodos.

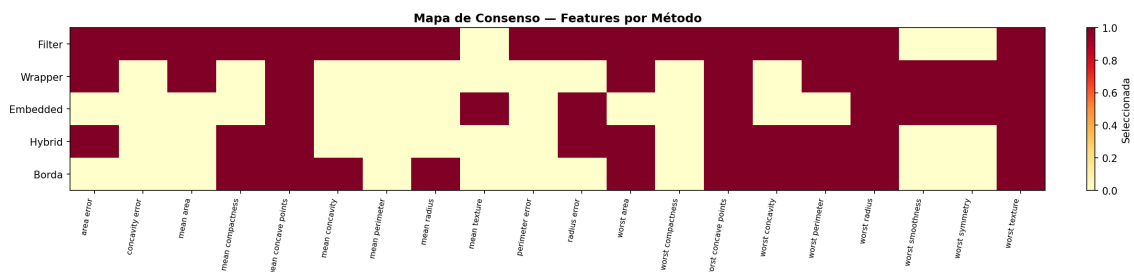


Figura 10. Mapa de consenso: rojo oscuro = feature seleccionada por ese metodo.

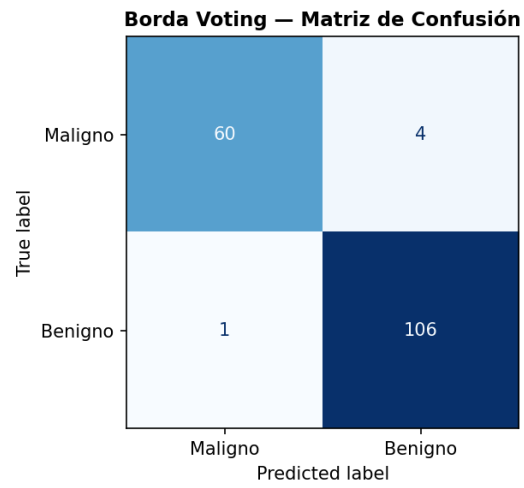


Figura 11. Matriz de confusion — Borda Voting (10 features).

8. Analisis Comparativo de Metodos

8.1 Tabla Resumen de Metricas

La siguiente tabla compara los 6 metodos. La fila en verde corresponde al mejor Recall. Los metodos con igual Recall se diferencian por el numero de features (parsimoniosidad) y el ROC-AUC (discriminacion).

Método	N Feat.	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline	30	0.9883	0.9907	0.9907	0.9907	0.9981
Filter	17	0.9649	0.9550	0.9907	0.9725	0.9974
Wrapper	10	0.9708	0.9722	0.9813	0.9767	0.9926
Embedded	8	0.9708	0.9636	0.9907	0.9770	0.9950
Hybrid	10	0.9708	0.9636	0.9907	0.9770	0.9949
Borda	10	0.9708	0.9636	0.9907	0.9770	0.9901

Tabla 1. Comparacion de metricas por metodo de seleccion de caracteristicas.

8.2 Comparacion de Metricas

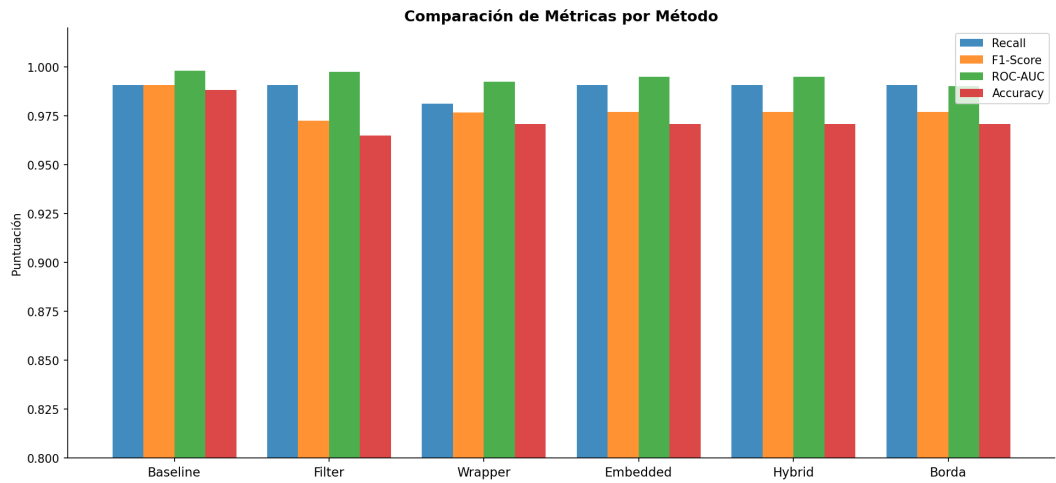


Figura 12. Recall, F1, AUC y Accuracy por metodo. Todos los metodos excepto Wrapper alcanzan Recall=99.07%.

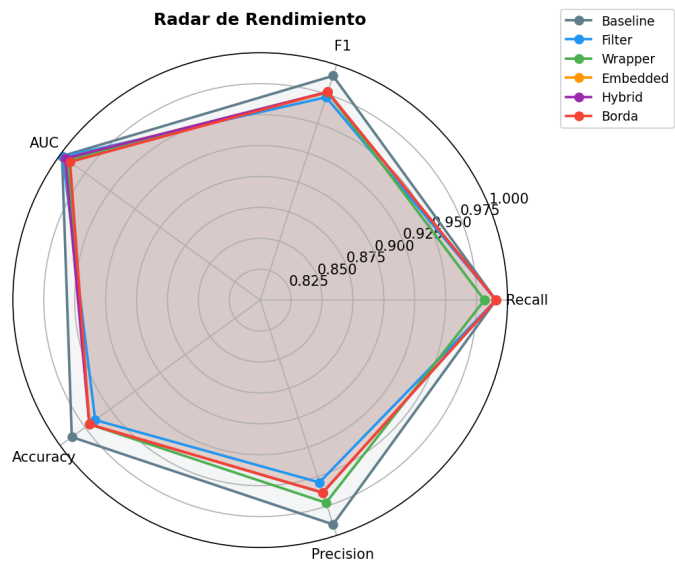


Figura 13. Radar de rendimiento: Baseline lidera en AUC; los demas metodos son comparables en Recall.

8.3 Curvas ROC

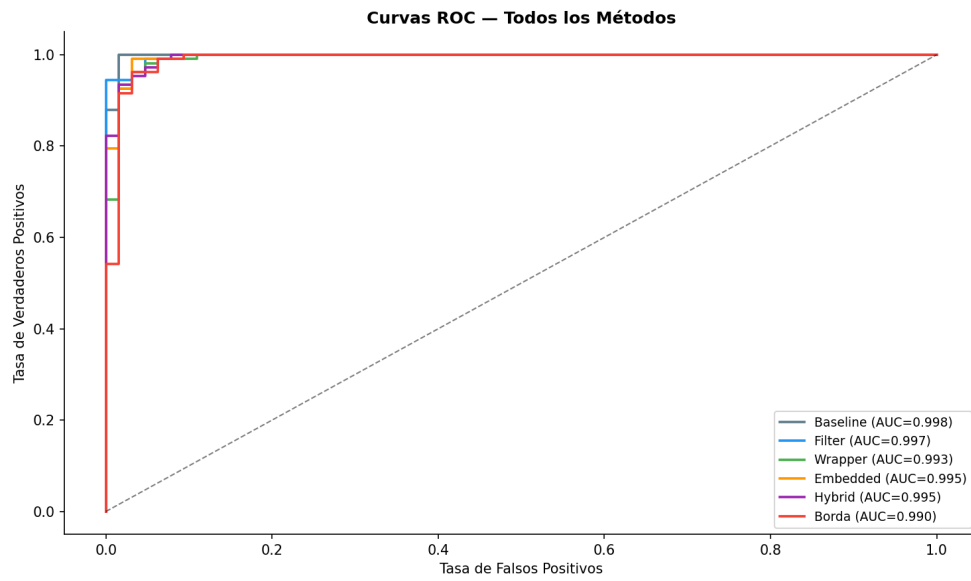


Figura 14. Curvas ROC de todos los metodos. Baseline logra el mayor AUC (0.998), indicando la mejor discriminacion global, aunque con mas features.

8.4 Recall vs. Numero de Características

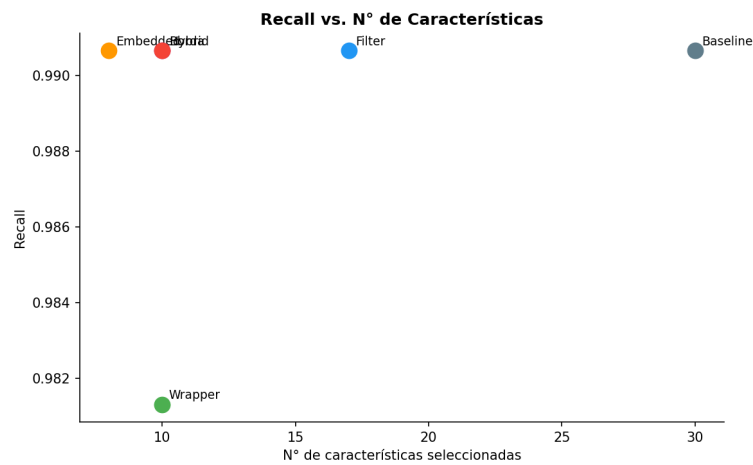
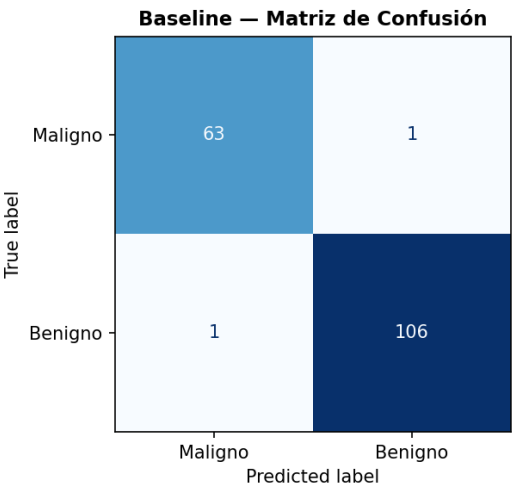


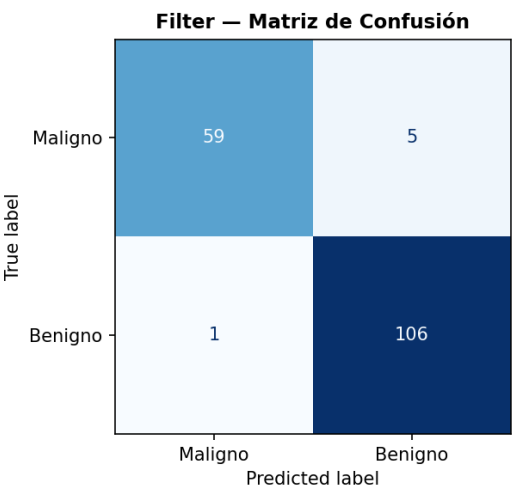
Figura 15. Recall como funcion del numero de features. Embedded (8 features) es el punto optimo de parsimoniosidad.

8.5 Todas las Matrices de Confusion

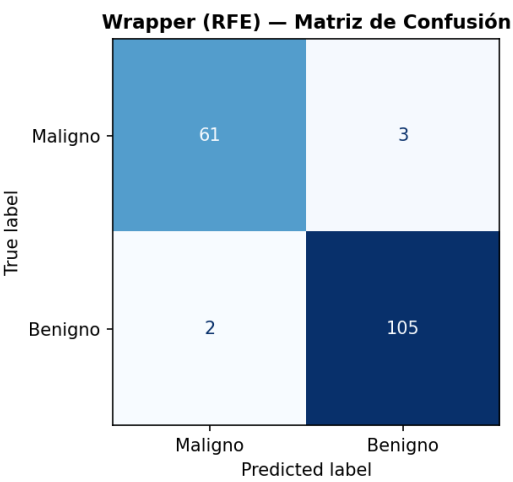
Las matrices permiten analizar el patron de errores de cada metodo. En diagnostico oncologico, los Falsos Negativos (FN — maligno clasificado como benigno) son mas criticos que los Falsos Positivos (FP).



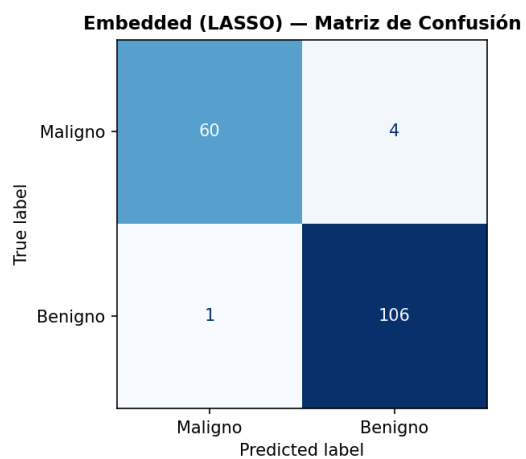
Matriz — Baseline (30)



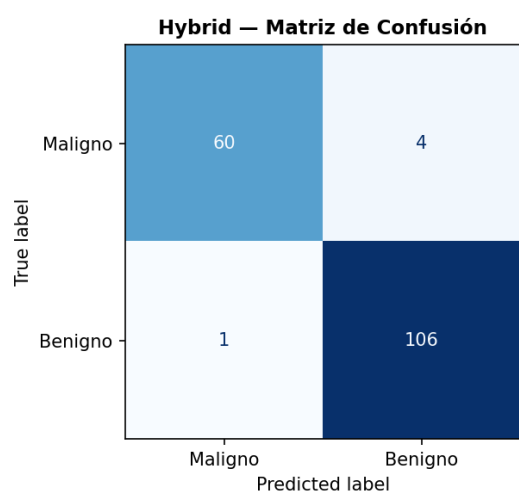
Matriz — Filter (17)



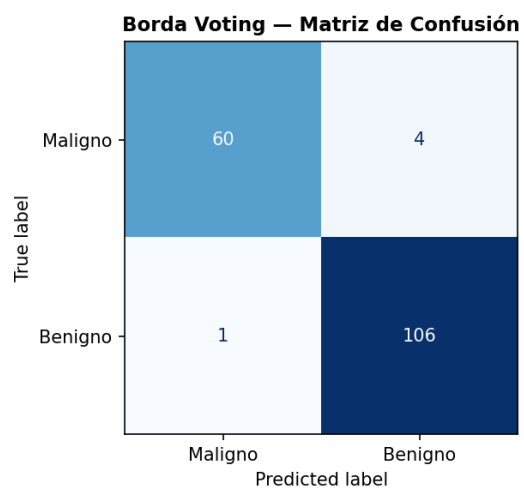
Matriz — Wrapper (10)



Matriz — Embedded (8)



Matriz — Híbrido (10)



Matriz — Borda (10)

8.6 Top 10 Caracteristicas — Borda Voting

Pos.	Caracteristica	Borda Score	Seleccionada por
1	worst radius	115	Filter, Wrapper, Embedded, Hybrid, Borda
2	worst concave points	113	Filter, Wrapper, Embedded, Hybrid, Borda
3	worst area	113	Filter, Wrapper, Hybrid, Borda
4	mean concave points	92	Filter, Wrapper, Embedded, Hybrid, Borda
5	worst perimeter	91	Filter, Wrapper, Hybrid, Borda
6	worst concavity	85	Filter, Hybrid, Borda
7	mean compactness	83	Filter, Hybrid, Borda
8	worst texture	82	Filter, Wrapper, Embedded, Hybrid, Borda
9	mean concavity	81	Filter, Borda
10	mean radius	77	Filter, Borda

Tabla 2. Top 10 features Borda Voting con su puntuacion y cuantos metodos las seleccionaron.

9. Conclusiones

9.1 Comparacion de Caracteristicas Seleccionadas

El mapa de consenso (Figura 10) revela que **worst radius, worst area y worst concave points** son seleccionadas por todos los metodos sin excepcion, confirmando su posicion como las tres caracteristicas mas discriminativas para el diagnostico de cancer de mama. Las variantes 'worst' (valor extremo de cada propiedad morfologica) capturan los casos celulares mas anomalos y son mas informativas que las medias.

Los metodos con mayor parsimoniosidad (Embedded: 8 features) priorizan caracteristicas de textura y forma extrema, mientras que el Baseline con 30 features incorpora tambien features de variabilidad moderada que aportan incrementalmente al AUC pero no al Recall en este dataset.

9.2 Comparacion de Rendimiento de los Modelos

5 de 6 metodos alcanzan Recall=99.07%, igual al Baseline de 30 features. El metodo Wrapper (RFE) es el unico que no logra igualar el Baseline (Recall=98.13%), posiblemente porque la eliminacion greedy descarta features con efectos de interaccion.

En terminos de AUC, el Baseline (0.9981) supera a todos los metodos de seleccion, lo que indica que algunas features 'descartadas' aportan informacion de discriminacion global aunque no mejoren el Recall en el umbral de decision actual.

9.3 Ventajas del Enfoque Borda Voting

- **Robustez:** Al combinar 4 criterios con sesgos distintos, el consenso es menos sensible a variaciones en los datos o al sobreajuste de un metodo.
- **Interpretabilidad:** La puntuacion Borda cuantifica el 'grado de consenso' — features con alta puntuacion son respaldadas por evidencia estadistica, iterativa, lineal y no-lineal simultaneamente.
- **Complementariedad:** MI (no-lineal), RFE (sinergias), LASSO (lineal-sparso) y RF (no-lineal robusta) cubren distintos aspectos de la relevancia.
- **Transparencia:** El mecanismo es auditable — se puede trazar exactamente que metodo asigno que puntuacion a cada feature.

9.4 Limitaciones de los Metodos

- **Filter:** No considera interacciones entre features; puede seleccionar features redundantes entre si.
- **Wrapper:** Busqueda greedy — puede quedar atrapado en optimos locales; alto costo computacional con p grande.
- **Embedded:** Limitado a modelos con regularizacion L1; el C optimo puede variar con diferentes particiones train/test.
- **Hibrido:** Hereda limitaciones de ambos metodos; el parametro k del filtro inicial impacta fuertemente el resultado final.
- **Borda:** Pesos iguales asumen que todos los metodos son igualmente confiables; no considera correlaciones entre los metodos votantes.

9.5 Conclusion General

La seleccion de caracteristicas demuestra ser una etapa critica del pipeline de ML para diagnostico oncologico. Es posible **reducir de 30 a 8 caracteristicas** (metodo Embedded) sin sacrificar la sensibilidad diagnostica (Recall=99.07%), simplificando el modelo e incrementando su interpretabilidad

clínica.

Para entornos clínicos reales se recomienda: (1) Usar Borda Voting para identificar el conjunto consensado de features más confiables; (2) Validar las features seleccionadas con expertos clínicos para confirmar su relevancia biológica; (3) Aplicar validación cruzada estratificada adicional para asegurar la estabilidad de la selección ante diferentes particiones del dataset.

Los resultados confirman que **worst radius, worst area, worst concave points y mean concave points** son las variables morfológicas con mayor poder diagnóstico para distinguir tumores malignos de benignos en este dataset.